

BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE PUEBLA (BUAP)

Programa Interinstitucional para el Fortalecimiento de la Investigación y el Posgrado del Pacífico — Programa Delfín

PROYECTO INTEGRADOR

EMPRENDEIA

Manual Técnico

Guía de arquitectura, configuración, despliegue y mantenimiento del sistema

LÍDER TÉCNICO

Isaías de Jesús Avilés Rodríguez

EQUIPO DE TRABAJO

Juan Carlos Gerónimo Pérez · Daniel Hernández Martínez
Daniel Ballesteros Rivera · América Alexandra Salazar Medina
Jorge Esteban Frayre Arceo · Javier Antonio Adrián Rivero

APROBADO POR

Dr. Jesús Andrés Arzola Flores

México · Julio de 2026

Contenido

1. Introducción	4
1.1 Alcance del documento	4
1.2 Repositorio y demo	4
2. Arquitectura del sistema	5
2.1 Capas	5
2.2 Diagrama lógico de capas	5
2.3 Principios de diseño aplicados	6
3. Stack tecnológico	7
3.1 Dependencias declaradas (pom.xml)	7
4. Estructura del proyecto	8
4.1 Responsabilidad de cada paquete	8
5. Seguridad y autenticación	9
5.1 Flujo de autenticación	9
5.2 Almacenamiento de contraseñas	9
5.3 Aislamiento de datos	9
5.4 Gestión de sesión	9
6. Modelo de datos	10
6.1 Entidades principales	10
6.2 Tipos de análisis	10
6.3 Los tres conceptos de estado	11
7. Rutas y controladores	12
7.1 Autenticación e inicio	12
7.2 Ideas de negocio	12
7.3 Análisis, finanzas y reporte	12
8. Integración con el modelo de lenguaje	13
8.1 Selección del proveedor	13
8.2 Construcción del prompt	13
8.3 Validación de la respuesta	13
8.4 Robustez de la llamada	13
9. Módulo financiero	14
9.1 Fórmulas	14
9.2 Validaciones	14
10. Generación del reporte ejecutivo	15
10.1 Estados del reporte	15
10.2 Degradación controlada	15

10.3	Aviso financiero.....	15
11.	Instalación y ejecución en local	16
11.1	Requisitos previos	16
11.2	Preparación de la base de datos.....	16
11.3	Variables de entorno.....	16
11.4	Compilación y arranque	16
12.	Configuración de la aplicación	17
13.	Despliegue en contenedor	18
13.1	Despliegue en Render	18
14.	Pruebas y verificación	19
14.1	Recomendaciones de verificación manual	19
15.	Mantenimiento y extensión.....	19
15.1	Añadir un proveedor de IA.....	19
15.2	Buenas prácticas del repositorio.....	19

1. Introducción

EmprendeIA es una aplicación web que emplea modelos de lenguaje (LLM) para asistir a emprendedores, estudiantes y microempresarios en la creación, análisis y validación inicial de planes de negocio. A partir de una idea capturada mediante formularios guiados, el sistema genera un diagnóstico empresarial, un modelo Canvas, un análisis FODA, un plan de marketing, un módulo financiero con interpretación asistida por IA y un reporte ejecutivo descargable en PDF.

Este Manual Técnico está dirigido al equipo de desarrollo y a cualquier persona que deba instalar, ejecutar, mantener o extender el sistema. Describe la arquitectura, el stack tecnológico, el modelo de datos, la configuración, el flujo de las peticiones y el procedimiento de despliegue, con base en el código fuente del repositorio del proyecto.

1.1 Alcance del documento

El manual cubre la instalación en entorno local, la configuración por variables de entorno, la estructura del proyecto Spring Boot, la integración con el proveedor de LLM, la generación del reporte PDF y el despliegue en contenedor. No sustituye a la Especificación de Requisitos de Software (SRS), que define el detalle de cada requisito funcional y no funcional; ambos documentos son complementarios.

1.2 Repositorio y demo

Recurso	Ubicación
Repositorio	github.com/IsaiasDJAviles/EmprendeIA
Demo en línea	https://emprendeia.onrender.com
groupId Maven	com.emprendeia
Artefacto	emprendeia-0.0.1-SNAPSHOT.jar

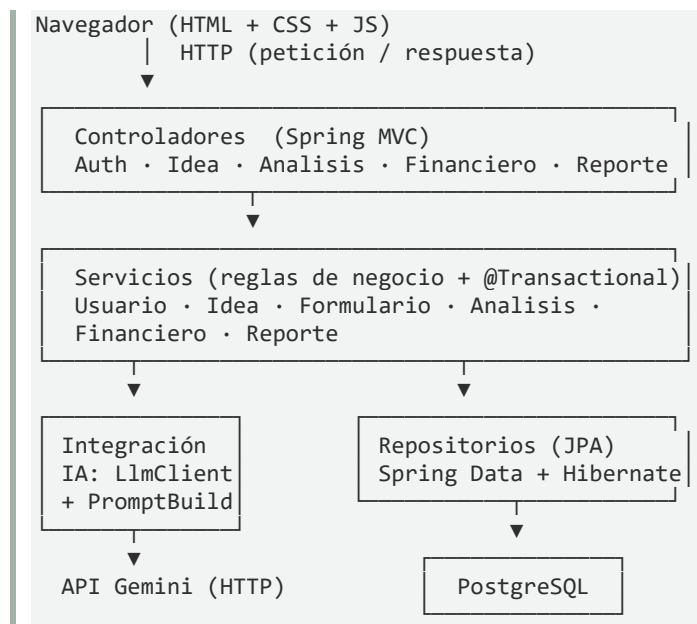
2. Arquitectura del sistema

El sistema implementa una arquitectura web en capas basada en el patrón Modelo–Vista–Controlador (MVC) sobre Spring Boot. La comunicación con el usuario es de tipo petición–respuesta sobre HTTP; las vistas se renderizan en el servidor con Thymeleaf. La inteligencia del sistema reside en la capa de servicio, que orquesta las llamadas al modelo de lenguaje a través de una interfaz de abstracción.

2.1 Capas

- **Capa de presentación (frontend):** vistas HTML renderizadas con Thymeleaf en el servidor, estilizadas con CSS y con JavaScript del lado del cliente para validación e interactividad. Un fragmento común de navegación se comparte entre vistas.
- **Capa de aplicación (controladores):** controladores Spring MVC que atienden las peticiones HTTP, resuelven vistas Thymeleaf y, en el caso del reporte, exponen la descarga binaria del PDF.
- **Capa de servicio (lógica de negocio):** servicios que encapsulan la lógica de cada módulo, aplican las reglas de validación, controlan la propiedad de los datos por usuario y gestionan las transacciones.
- **Capa de integración con IA:** la interfaz LlmClient define el puerto hacia el proveedor; PromptBuilder construye los prompts parametrizados; la implementación concreta envía la solicitud HTTP y deserializa la respuesta JSON.
- **Capa de generación de reportes:** PdfReportBuilder arma el PDF del reporte ejecutivo con OpenPDF a partir de los datos ya persistidos.
- **Capa de datos (persistencia):** repositorios de Spring Data JPA sobre Hibernate, con PostgreSQL como SGBD. Hibernate opera en modo validate: verifica el esquema contra las entidades pero nunca lo modifica.

2.2 Diagrama lógico de capas



2.3 Principios de diseño aplicados

- **Abstracción del proveedor de IA.** El resto de la aplicación depende de la interfaz LlmClient, no de un proveedor concreto. Cambiar de proveedor es una cuestión de configuración, no de reescritura de código.
- **Configuración externalizada.** Credenciales, URL base, modelo y proveedor viven en variables de entorno; nunca están escritos en el código.
- **Aislamiento de datos por usuario.** Toda operación sobre una idea pasa por una verificación de propiedad; una idea ajena responde igual que una inexistente para no revelar su existencia por enumeración de identificadores.
- **Disciplina de esquema.** Hibernate valida el esquema (ddl-auto=validate) y el rol de aplicación carece de privilegios DDL, de modo que la aplicación nunca altera la estructura de la base de datos en tiempo de ejecución.
- **Degradación controlada.** El reporte se genera aunque falten módulos: las secciones ausentes se marcan como pendientes en vez de bloquear todo el documento.

3. Stack tecnológico

Componente	Tecnología	Función
Lenguaje	Java 24	Lenguaje base del backend.
Framework	Spring Boot 4.1.0	Núcleo de la aplicación: MVC, inyección de dependencias, autoconfiguración.
Web	Spring Web MVC	Controladores y resolución de vistas.
Vistas	Thymeleaf	Renderizado de HTML en el servidor.
Seguridad	Spring Security + BCrypt	Autenticación por formulario y hashing de contraseñas.
Persistencia	Spring Data JPA / Hibernate	Mapeo objeto-relacional y repositorios.
Validación	Jakarta Bean Validation	Validación declarativa de formularios y propiedades.
Base de datos	PostgreSQL	Sistema gestor de base de datos relacional.
IA	Google Gemini API	Generación de contenido de los módulos de análisis.
Reportes	OpenPDF 2.2.2	Construcción del reporte ejecutivo en PDF.
JSON	Jackson	Serialización y validación de las respuestas del LLM.
Empaque/despliegue	Maven + Docker	Compilación reproducible e imagen de contenedor.

3.1 Dependencias declaradas (pom.xml)

El proyecto hereda de spring-boot-starter-parent en la versión 4.1.0 y declara los starters de Data JPA, Security, Thymeleaf, Validation, Web MVC y JSON, además de la extensión de Thymeleaf para Spring Security, el driver de PostgreSQL (en ámbito runtime) y OpenPDF. Las dependencias de prueba incluyen los starters de test para JPA, Security, Thymeleaf, Validation y Web MVC.

4. Estructura del proyecto

El código se organiza por responsabilidad bajo el paquete raíz `com.emprendeia`. Cada paquete agrupa una preocupación específica:

```
com.emprendeia
├── EmprendeiaApplication      // arranque Spring Boot
├── config/                   // SecurityConfig
├── controller/               // 5 controladores MVC
├── dto/                      // formularios y payloads validados
├── model/                    // entidades JPA (15 tablas)
├── repository/               // repositorios Spring Data
├── service/                  // lógica de negocio + transacciones
├── security/                 // UserDetailsService + Principal
├── ia/                       // LlmClient, PromptBuilder, LlmException
│   ├── gemini/               // GeminiClient, GeminiProperties
├── report/                   // PdfReportBuilder, DatosReporte
├── exception/                // excepciones de dominio

src/main/resources
├── application.properties    // configuración externalizada
├── static/css, static/js     // estilos y scripts del cliente
└── templates/                // vistas Thymeleaf
```

4.1 Responsabilidad de cada paquete

Paquete	Contenido y responsabilidad
config	Configuración de Spring Security: cadena de filtros, rutas públicas, login por formulario, codificador BCrypt y proveedor de autenticación.
controller	Puntos de entrada HTTP. Un controlador por área funcional: autenticación, ideas, análisis, financiero y reporte.
dto	Objetos de transferencia con anotaciones de validación. Separan la forma del formulario de la entidad persistente.
model	Entidades JPA mapeadas a las 15 tablas del esquema, con nombres de columnas en <code>snake_case</code> .
repository	Interfaces Spring Data JPA; una consulta personalizada para la búsqueda de ideas activas por usuario.
service	Reglas de negocio, control de propiedad por usuario, cálculo financiero y fronteras transaccionales.
security	Carga del usuario por correo y adaptación al contrato <code>UserDetails</code> de Spring Security.
ia	Abstracción del proveedor de LLM, construcción de prompts y contrato de campos esperados.
report	Recolección de datos y construcción del PDF del reporte ejecutivo.
exception	Excepciones de dominio que los controladores traducen a redirecciones con mensaje.

5. Seguridad y autenticación

La seguridad se configura en SecurityConfig mediante la cadena de filtros de Spring Security. Las rutas públicas comprenden la página de inicio, el registro, el login, la página de error y los recursos estáticos (CSS, JS, imágenes y webjars). Cualquier otra ruta exige autenticación.

5.1 Flujo de autenticación

1. El usuario envía el formulario de login con los campos **correo y contraseña**.
2. UsuarioDetailsService carga el usuario por correo y lo adapta a UsuarioPrincipal.
3. DaoAuthenticationProvider compara la contraseña con el hash BCrypt almacenado.
4. En caso de éxito, la sesión se establece y se redirige al panel principal; en caso de fallo, se regresa al login con un indicador de error.

5.2 Almacenamiento de contraseñas

Las contraseñas nunca se guardan en texto plano. Al registrar un usuario, el servicio codifica la contraseña con BCrypt antes de persistirla; la columna correspondiente reserva 60 caracteres, el tamaño del hash BCrypt.

5.3 Aislamiento de datos

Cada acceso a una idea de negocio se resuelve a través de un método que verifica la propiedad del recurso por el usuario en sesión. Si la idea no existe o pertenece a otro usuario, se lanza la misma excepción con idéntico mensaje, evitando revelar por enumeración de identificadores si una idea ajena existe.

5.4 Gestión de sesión

El tiempo de expiración de sesión es configurable mediante una variable de entorno, con un valor predeterminado de 30 minutos.

6. Modelo de datos

El esquema comprende 15 tablas con nomenclatura en snake_case y llaves primarias autoincrementales. Se distinguen tres conceptos de estado que no deben confundirse: el estatus técnico (soft delete), el estado de negocio del reporte y el estado geográfico del catálogo. Las tablas transaccionales usan borrado lógico; los catálogos no.

6.1 Entidades principales

Entidad / tabla	Descripción
usuario	Personas registradas. Correo único, contraseña con hash BCrypt, fecha de registro y estatus.
idea_negocio	Idea capturada por el usuario: nombre, descripción, problema, sector, cliente objetivo, tipo de oferta, nivel de avance y estado geográfico.
formulario	Datos financieros de entrada (1:1 con la idea): inversión inicial, costos fijos y variables, precio de venta, unidades estimadas y destino de la inversión.
analisis_generado	Resultado de un módulo de IA: tipo de análisis, contenido JSON, prompt completo y fecha de generación. Un registro por idea y tipo.
modulo_financiero	Resultado calculado en Java: ingresos estimados, costos totales y variables, utilidad y punto de equilibrio.
reporte	Reporte ejecutivo de la idea, con su estado de negocio (borrador, consolidado, descargado) y fecha.
estatus	Catálogo de estatus técnico (por ejemplo, ACTIVO / INACTIVO) para el borrado lógico.
estado_reporte	Catálogo del estado de negocio del reporte.
pais, estado	Catálogos geográficos, alcance País y Estado para Latinoamérica.
giro, tipo_capital	Catálogos enlazados a la idea mediante tablas intermedias (relación N:M).
socio	Normaliza los socios de la idea; cubre también el equipo de gestión.

6.2 Tipos de análisis

El tipo de análisis se modela como una enumeración con cinco valores. Cada valor determina el prompt, la forma del JSON esperado y la vista de edición correspondiente.

Valor	Módulo funcional
DIAGNOSTICO	Diagnóstico empresarial inicial.
CANVAS	Modelo Canvas de nueve bloques.
FODA	Análisis de fortalezas, oportunidades, debilidades y amenazas.
MARKETING	Plan de marketing inicial.
MERCADO	Ocupa el slot de la interpretación financiera asistida por IA (ver sección 8).

Nota de diseño. El valor MERCADO reutiliza el slot de la enumeración existente para alojar la interpretación financiera, en lugar de agregar un valor nuevo a la restricción CHECK de la tabla de análisis. El LLM interpreta cifras ya calculadas con precisión por el servicio financiero; nunca las recalcula.

6.3 Los tres conceptos de estado

Concepto	Columna	Propósito
Estatus técnico	id_estatus	Borrado lógico (soft delete) de registros transaccionales.
Estado del reporte	id_estado_reporte	Flujo de negocio del reporte: borrador, consolidado, descargado.
Estado geográfico	id_estado	Referencia al catálogo geográfico (país/estado).

7. Rutas y controladores

La siguiente tabla resume los puntos de entrada HTTP expuestos por los controladores, agrupados por área funcional. Las rutas bajo /ideas requieren sesión y verifican la propiedad del recurso.

7.1 Autenticación e inicio

Método	Ruta	Acción
GET	/	Página de inicio (landing).
GET	/registro	Formulario de registro.
POST	/registro	Alta de usuario con validación y hash BCrypt.
GET	/login	Formulario de inicio de sesión.
POST	/login	Autenticación (gestionada por Spring Security).
POST	/logout	Cierre de sesión.

7.2 Ideas de negocio

Método	Ruta	Acción
GET	/dashboard	Panel principal; lista, busca y ordena las ideas del usuario.
GET	/ideas/nueva	Formulario de nueva idea.
POST	/ideas/nueva	Crea la idea.
POST	/ideas/{id}/eliminar	Borrado lógico de la idea.
GET	/ideas/{id}/formulario	Formulario guiado de datos financieros.
POST	/ideas/{id}/formulario	Guarda los datos del formulario guiado.

7.3 Análisis, finanzas y reporte

Método	Ruta	Acción
GET	/ideas/{id}/analisis/{tipo}	Muestra el módulo (diagnóstico, canvas, foda, mercado, marketing).
POST	/ideas/{id}/analisis/{tipo}/generar	Genera el módulo con el LLM.
POST	/ideas/{id}/analisis/{tipo}	Guarda la edición manual del módulo.
GET	/ideas/{id}/finanzas	Muestra el módulo financiero y el resultado.
POST	/ideas/{id}/finanzas/calcular	Calcula finanzas y genera la interpretación IA.
GET	/ideas/{id}/reporte	Muestra el estado del reporte por secciones.
POST	/ideas/{id}/reporte/generar	Genera o actualiza el reporte.
GET	/ideas/{id}/reporte/descargar	Construye y descarga el PDF.

8. Integración con el modelo de lenguaje

La integración con IA se apoya en tres piezas: la interfaz LlmClient (puerto hacia el proveedor), PromptBuilder (construcción de prompts y contrato de campos esperados) y la implementación concreta GeminiClient. El resto de la aplicación depende solo de la interfaz.

8.1 Selección del proveedor

La implementación activa se selecciona por la propiedad llm.provider. GeminiClient se activa cuando el valor es gemini (también por defecto si la propiedad no está presente). Para incorporar otro proveedor basta con añadir una implementación de LlmClient en su propio subpaquete y activarla por configuración, sin tocar los servicios.

8.2 Construcción del prompt

PromptBuilder arma un prompt por tipo de análisis a partir de la información de la idea y, en el caso de la interpretación financiera, de las cifras ya calculadas. Cada prompt termina con una instrucción que exige responder únicamente con un JSON válido y con la estructura de campos esperada.

8.3 Validación de la respuesta

El servicio de análisis parsea la respuesta como JSON y verifica que estén presentes todos los campos esperados para ese tipo. Si el proveedor no devuelve JSON válido o falta algún campo obligatorio, se lanza una excepción de análisis inválido antes de persistir nada, de modo que el usuario puede reintentar sin perder información.

8.4 Robustez de la llamada

- El cliente HTTP fija un tiempo de espera de conexión y lectura de 30 segundos.
- La generación no se ejecuta dentro de una transacción de base de datos: la llamada al LLM puede tardar y no debe mantener una conexión ociosa; la escritura final es una única operación transaccional por sí misma.
- Los errores HTTP del proveedor y los fallos de red se traducen a una excepción de dominio con mensaje claro.
- Para la interpretación financiera, si aún no se han calculado las finanzas, la generación falla de inmediato antes de llamar al LLM.

9. Módulo financiero

El cálculo financiero se realiza en Java puro, sin intervención del LLM, a partir de los datos del formulario guiado. El servicio deriva ingresos estimados, costos totales, utilidad, margen de contribución y punto de equilibrio, empleando aritmética decimal de precisión para evitar errores de redondeo.

9.1 Fórmulas

Magnitud	Cálculo
Ingresos estimados	$\text{precio de venta} \times \text{unidades estimadas}$
Costos totales	$\text{costos fijos} + \text{costos variables}$
Utilidad	$\text{ingresos estimados} - \text{costos totales}$
Costo variable unitario	$\text{costos variables} \div \text{unidades estimadas}$
Margen de contribución unitario	$\text{precio de venta} - \text{costo variable unitario}$
Margen de contribución porcentual	$\text{margen de contribución unitario} \div \text{precio de venta}$
Punto de equilibrio	$\text{costos fijos} \div \text{margen de contribución porcentual}$

9.2 Validaciones

- Si las unidades estimadas no son mayores que cero, no es posible calcular el punto de equilibrio y se rechaza la operación.
- Si el precio de venta no cubre el costo variable unitario (margen de contribución no positivo), tampoco es posible el cálculo y se informa al usuario.

Persistencia parcial. La tabla del módulo financiero no tiene columna para el margen de contribución: ese valor viaja en el resultado que se muestra en pantalla, pero se recalcula en cada solicitud a partir del formulario vigente y no se almacena.

10. Generación del reporte ejecutivo

El reporte consolida el diagnóstico, el modelo Canvas, el FODA, el plan de marketing, la tabla financiera y la interpretación financiera con IA en un único documento PDF. La construcción del PDF se realiza al vuelo en cada descarga, ya que no se almacenan los bytes del documento en la base de datos; sí se persiste el estado de negocio del reporte.

10.1 Estados del reporte

Estado	Significado
borrador	El reporte existe pero faltan módulos por completar.
consolidado	Todos los módulos de análisis y las finanzas están disponibles.
descargado	El usuario ya descargó el PDF al menos una vez.

10.2 Degradación controlada

Un análisis faltante, un formulario sin llenar o unas finanzas sin calcular no bloquean la generación del reporte: cada hueco se renderiza como un aviso de "pendiente" en el PDF y el reporte se marca como borrador en lugar de consolidado. Así, el fallo puntual de un módulo de IA no impide obtener el resto del documento.

10.3 Aviso financiero

El PDF incluye un aviso que aclara que los cálculos financieros son estimaciones a partir de la información proporcionada por el usuario, que no constituyen asesoría financiera profesional y que conviene validarlos con un especialista antes de tomar decisiones de inversión.

11. Instalación y ejecución en local

11.1 Requisitos previos

- Java 24 (JDK) instalado y disponible en el PATH.
- PostgreSQL en ejecución, con una base de datos para la aplicación.
- Una API key de Gemini, obtenida en Google AI Studio (<https://aistudio.google.com/apikey>).

11.2 Preparación de la base de datos

Cree la base de datos y ejecute, en orden, los scripts de esquema y de roles incluidos en el directorio db del proyecto. El primero crea las 15 tablas y sus catálogos; el segundo crea los roles de base de datos con privilegios acotados.

```
# Ejecutar en orden:
psql -d emprendeia -f db/EmprendeIA_BD.sql
psql -d emprendeia -f db/emprendeia_roles.sql
```

Precondición de datos. El catálogo de estatus debe contener las filas ACTIVO e INACTIVO, y el catálogo de estado de reporte debe contener borrador, consolidado y descargado; varios servicios los buscan por nombre al operar.

11.3 Variables de entorno

Variable	Descripción	Valor por defecto
DB_HOST	Host de PostgreSQL.	localhost
DB_PORT	Puerto de PostgreSQL.	5432
DB_NAME	Nombre de la base de datos.	emprendeia
DB_APP_USERNAME	Usuario del rol de aplicación.	—
DB_APP_PASSWORD	Contraseña del rol de aplicación.	—
DB_SSLMODE	Modo SSL de la conexión.	prefer
GEMINI_API_KEY	Clave de la API de Gemini.	—
PORT / SERVER_PORT	Puerto HTTP de la aplicación.	8080
SESSION_TIMEOUT	Expiración de sesión.	30m

11.4 Compilación y arranque

Con las variables exportadas, el arranque se realiza con el wrapper de Maven incluido en el repositorio:

```
./mvnw spring-boot:run          # ejecución directa
./mvnw clean package -DskipTests # genera el JAR ejecutable
java -jar target/emprendeia-0.0.1-SNAPSHOT.jar
```

Una vez iniciada, la aplicación queda disponible en <http://localhost:8080>.

12. Configuración de la aplicación

La configuración vive en `application.properties` y toma todos los valores sensibles de variables de entorno.

Los puntos clave son:

- **Puerto HTTP.** Se resuelve en cascada: primero `PORT` (que inyecta el proveedor de despliegue), luego `SERVER_PORT` y, en último término, `8080`.
- **Origen de datos.** La URL JDBC se compone de host, puerto, nombre de base y modo SSL; el usuario y la contraseña provienen del rol de aplicación y nunca están escritos en el archivo.
- **Hibernate en modo validate.** El rol de aplicación carece de privilegios DDL; Hibernate valida el esquema contra las entidades pero nunca lo modifica.
- **Proveedor de IA.** `Ilm.provider` selecciona el cliente; para Gemini se configuran el modelo, la URL base y la clave de API.

13. Despliegue en contenedor

El repositorio incluye un Dockerfile de construcción multietapa. La primera etapa usa una imagen JDK 24, resuelve las dependencias en una capa separada (para aprovechar la caché) y compila el JAR; la segunda etapa parte de una imagen JRE ligera y copia solo el artefacto final.

```
# Etapa de build: JDK 24, dependencias en caché, package sin tests
# Etapa de runtime: JRE 24, copia app.jar y expone el puerto 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

13.1 Despliegue en Render

El objetivo de despliegue es Render, por su integración nativa con GitHub y su base de datos PostgreSQL gestionada. Se configuran las mismas variables de entorno que en local, añadiendo `DB_SSLMODE=require` para forzar TLS en la conexión a la base de datos gestionada.

Caveat del plan gratuito. El plan gratuito suspende el servicio por inactividad; la primera petición tras un periodo inactivo experimenta un arranque en frío con latencia adicional. Es un comportamiento esperado, no un defecto de la aplicación.

14. Pruebas y verificación

El proyecto declara los starters de prueba de Spring Boot para JPA, Security, Thymeleaf, Validation y Web MVC. La verificación funcional durante el desarrollo se apoya en colecciones de Postman para ejercitar las rutas y en la revisión visual de las vistas y del PDF generado.

14.1 Recomendaciones de verificación manual

1. Registrar un usuario y verificar que la contraseña se almacena como hash BCrypt.
2. Crear una idea, completar el formulario guiado y calcular las finanzas; comprobar las validaciones de unidades y de margen.
3. Generar cada módulo de análisis y confirmar que la respuesta se persiste y se puede editar.
4. Generar y descargar el reporte con módulos completos e incompletos, verificando la degradación a "pendiente".

15. Mantenimiento y extensión

15.1 Añadir un proveedor de IA

1. Crear una clase que implemente LlmClient en un subpaquete propio dentro de ia.
2. Activarla de forma condicional según el valor de llm.provider.
3. Configurar sus propiedades (modelo, URL base y credencial) por variables de entorno.
4. Cambiar llm.provider al nuevo valor; ningún servicio requiere modificación.

15.2 Buenas prácticas del repositorio

- Estrategia de ramas Git Flow y pruebas de las rutas con colecciones de Postman.
- Modo validate de Hibernate como salvaguarda: cualquier divergencia entre entidades y esquema detiene el arranque.
- Credenciales y claves siempre en variables de entorno, nunca versionadas.